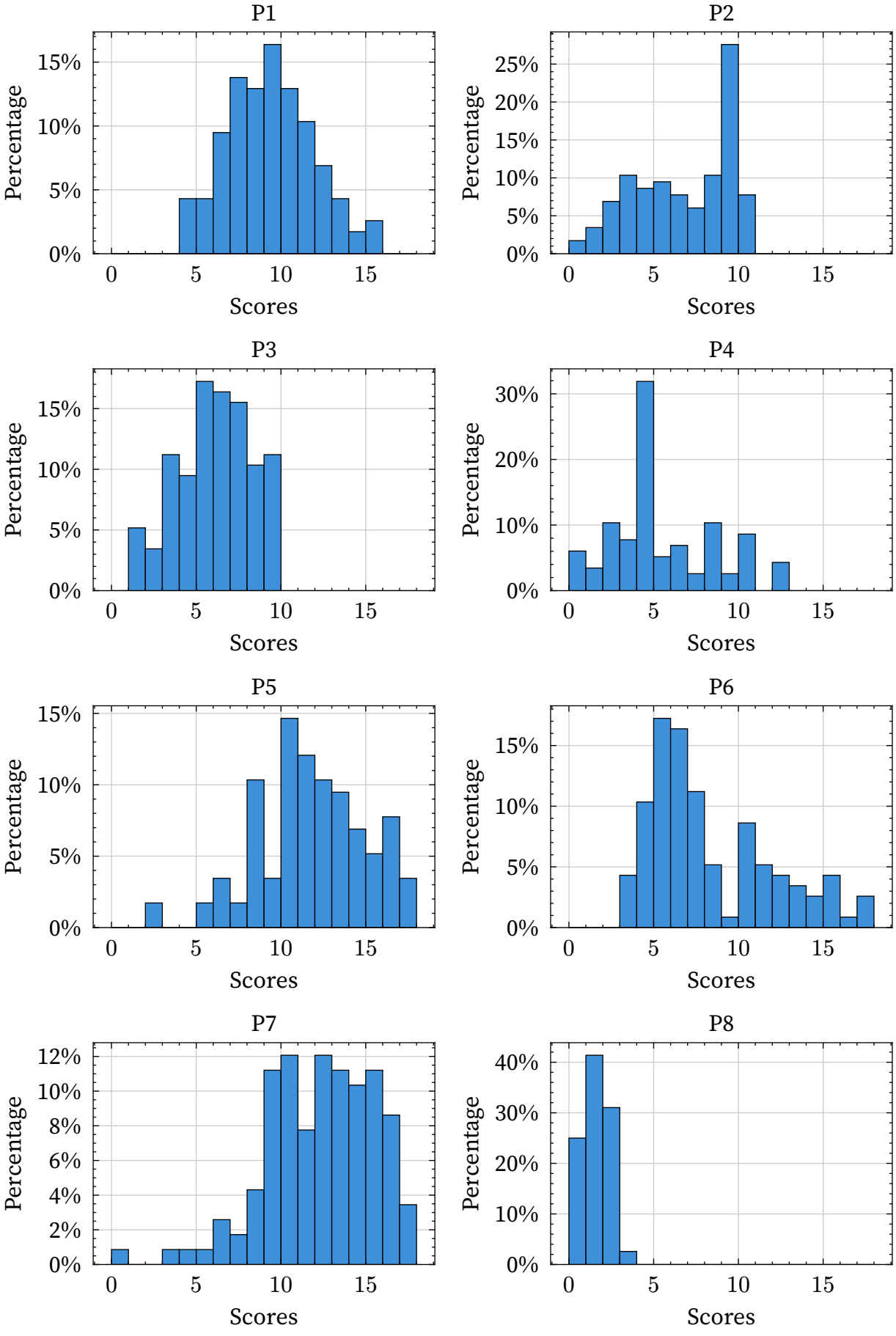
# SICP 期中考试小题统计数据

## 各题统计数据

| 题目 | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 |
|---|---|---|---|---|---|---|---|---|
| **分值平均值** | 8.88 | 6.28 | 5.6 | 5.04 | 11.87 | 8.11 | 11.87 | 1.11 |
| **分值中位数** | 9 | 7 | 6 | 4 | 12 | 7 | 12 | 1 |
| **得分率平均值** | 59.2% | 62.76% | 62.26% | 42.03% | 62.48% | 45.07% | 69.83% | 37.07% |
| **得分率标准差** | 0.17 | 0.28 | 0.24 | 0.26 | 0.19 | 0.22 | 0.19 | 0.27 |

Table 1: 各题平均得分率与其标准差

## 各题分值分布情况


P1
Percentage
15%
10%
5%
0%
0
5
10
15
Scores
P2
Percentage
25%
20%
15%
10%
5%
0%
0
5
10
15
Scores
P3
Percentage
15%
10%
5%
0%
0
5
10
15
Scores
P4
Percentage
30%
20%
10%
0%
0
5
10
15
Scores
P5
Percentage
15%
10%
5%
0%
0
5
10
15
Scores
P6
Percentage
15%
10%
5%
0%
0
5
10
15
Scores
P7
Percentage
12%
10%
8%
6%
4%
2%
0%
0
5
10
15
Scores
P8
Percentage
40%
30%
20%
10%
0%
0
5
10
15
Scores

# 期中考试题 1、题 2 解析

## 题 1 - WWPD

### 评分标准

1. 共 15 个输出，每个输出一分；
2. 对于每一部分，只要多写，就扣一分。

因此可能出现一个部分中明明打了一个勾，但是由于多写导致这一分被扣除了，结果是没有分。

### 第一部分 - REPL 和 `print` 的陷阱

#### 题目

```Python
print
print("hello") and print("world")
```

#### 答案

```output
Function
hello
```

1. `print` 是个函数，所以第一行回答 `Function`。
2. `print("hello") and print("world")`
   1. 遇到 `and` 表达式，先对左边求值；
   2. 对左边 `print("hello")` 求值，打印 `hello`；
   3. `print` 打印字符串不会加引号；
   4. `print("hello")` 返回 `None`；
   5. `None` 不为真，`and` 表达式左边不为真，就返回左边表达式的值（而非 `False`）；
   6. 因此整个 `and` 表达式的值为 `None`；
   7. REPL 遇到值为 `None` 的表达式不输出任何内容。

### 第二部分 - 拷贝与易错的 mutation

#### 题目

```Python
lst1 = [1, 2, 3]
lst2 = lst1 * 2
lst3 = [lst1] * 2
lst1[1] = 10

lst1
lst2
lst3
```

**答案**

```
[1, 10, 3]
[1, 2, 3, 1, 2, 3]
[[1, 10, 3], [1, 10, 3]]
```
output

本题类似 lab04 - Problem 3。

1. `lst2 = lst1 * 2` 创建一个列表赋值给 `lst2`，内容是将 `lst1` 中的元素重复两遍。
2. `[lst1]` 是一个列表，其中只有一个元素，就是 `lst1` 对象，或者可以说是它的引用。`[lst1][0]` 和 `lst1` 是同一个对象，`[lst1][0] is lst1`。
3. `lst3 = [lst1] * 2` 创建一个列表赋值给 `lst3`，内容是将 `[lst1]` 中的元素重复两遍。也就是把 `lst1` 这个引用重复两遍，结果就是 `lst3[0] is lst3[1] is lst1`。
4. `lst1[1] = 10`，将 `lst1` 的**第二个**元素修改为 `10`。
5. 修改 `[lst1]`，因为 `lst3[0] is lst3[1] is lst1`，所以这个修改会反映到 `lst3` 上。

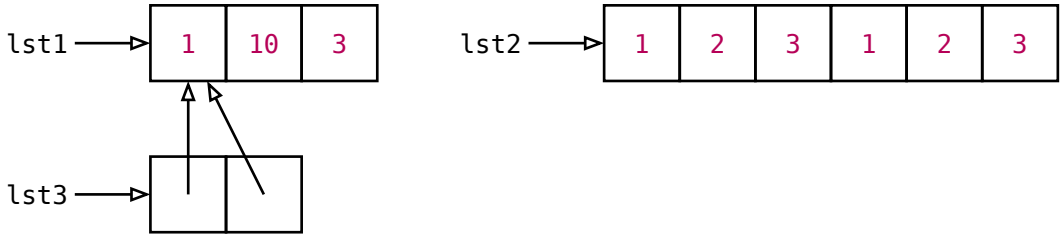



Figure 1: 三个列表的最终状态

**第三部分 - `range` 与 slicing**

**题目**

```
lst = list(range(1, 5))

lst
lst[1:-1]
```
Python

**答案**

```
[1, 2, 3, 4]
[2, 3]
```
output

1. 在 Python 中，不论是 `range` 还是列表的 slicing 都是左闭右开区间，因此 `list(range(1, 5)) = [1, 2, 3, 4]`。
2. 在 Python 的 slicing 中，`-1` 指的是最后一个元素，因此 `[1:-1]` 表示从**第二个**元素开始，到最后一个元素前，左闭右开的切片。所以 `lst[1:-1] = [2, 3]`。

**第四部分 - 递归调用的计数**

**题目**

```
def counted_fib(n):
    count = 0
```
Python

```
    def fib(n):
        nonlocal count
        count += 1
        if n <= 1:
            return n
        else:
            return fib(n - 1) + fib(n - 2)
    result = fib(n)
    return (count, result)

counted_fib(0)
counted_fib(1)
counted_fib(2)
counted_fib(3)
```

**答案**

output
```
(1, 0)
(1, 1)
(3, 1)
(5, 2)
```

本题给出的 `counted_fib` 函数就是在一个递归的 `fib` 函数上用 `nonlocal` 加了一个计数，没有设计其他的坑点，包括名字也是明晃晃地地写着 `counted_fib`。

所以就是需要写出 `fib` 函数在前四项输入下分别需要被（递归）调用多少次才能算出结果。记 `fib(n)` 调用 `fib` 函数 $C_n$ 次。

1. `n = 0` 和 `n = 1` 显然需要调用 1 次，$C_0 = C_1 = 1$。
2. 对 `n >= 2`，需要调用 $C_n = 1 + C_{n-1} + C_{n-2}$ 次。

**第五部分 - 递归生成器之谜**

**题目**

Python
```
def mystery():
    yield 1
    yield 1
    iter1 = mystery()
    iter2 = mystery()
    next(iter2)
    yield from map(lambda pair: pair[0] + pair[1], zip(iter1, iter2))

series1 = mystery()
series2 = filter(lambda n: n % 2 != 0, series1)

```

```
12  [next(series1) for _ in range(2)]
13  [next(series1) for _ in range(2)]
14  [next(series2) for _ in range(2)]
15  next(series1)
```

**答案**

output
```
1  [1, 1]
2  [2, 3]
3  [5, 13]
4  21
```

对于第一小问 `[next(series1) for _ in range(2)]` 可以通过 `mystery` 的定义直接回答 `[1, 1]`。但后续的小问都需要大家能**看明白或者猜出** `mystery` 生成的数的含义。

**解谜**

我们来逐行看 `mystery`：

1. 首先 `yield` 了前两个元素，都是 `1`。
2. 然后递归调用了 `mystery`，获得 `iter1` 和 `iter2`，目前我们只知道这两个生成器内容一定和 `mystery` 相同，且前两个元素，都是 `1`。
3. 调用 `next(iter2)`，丢弃了 `iter2` 开头的一个元素，这意味着 `iter2` 比 `iter1` 落后一个元素。
4. 想得多的同学可能会意识到，当前调用的 `mystery` 比 `iter1` 快两个元素，比 `iter2` 快一个元素，所以这个函数在做什么，即使不看最后一样好像都呼之欲出了。
5. 但是没想到也没关系，我们继续看最后一行，它首先将 `iter1` 和 `iter2` `zip` 起来，变成一个生成器，内容是原先两个生成器元素的 pair。
6. `map` 将 pair 的两个元素加起来，也就是说把 `iter1` 和 `iter2` 中的元素逐个相加。

看完了，如果还是没有意识到这个函数生成的是斐波那契数列，我们就继续思考几项：

1. 第 3 项是 `iter1` 的第 1 项 `1` + `iter2` 的第 2 项 `1`，等于 `2`。
2. 第 4 项是 `iter1` 的第 2 项 `1` + `iter2` 的第 3 项。那么 `iter2` 的第 3 项等于什么呢，因为 `iter2 = mystery()`，我们已经知道 `mystery()` 的第三项是 `2`，所以 `iter2` 的第 3 项就为 `2`。所以 `mystery()` 的第四项等于 `3`。
3. 第 5 项是 `iter1` 的第 3 项 + `iter2` 的第 4 项。换句话说就是 `mystery()` 的第 3 项 + `mystery()` 的第 4 项。

这样就能看出来了，`mystery()` 的第 $n$ 项（$n \geq 2$ 时）就是 `mystery()` 的第 $n-2$ 项 + `mystery()` 的第 $n-1$ 项。`mystery()` 生成的就是斐波那契数列。

**答题**

解谜之后就能答剩下的题了。

Python
```
1  series1 = mystery()
2  series2 = filter(lambda n: n % 2 != 0, series1)
3
```

```
4 [next(series1) for _ in range(2)]
5 [next(series1) for _ in range(2)]
6 [next(series2) for _ in range(2)]
7 next(series1)
```

题目首先构造了两个生成器，首先是调用 `mystery()` 获得 `series1`，然后是对 `series1` 调用 `filter`，筛选出 `series1` 中的奇数，获得 `series2`。

此时，不论是 `series1` 还是 `series2` 都还没有生成任何元素。前两个小问连续的两个 `[next(series1) for _ in range(2)]` 分别输出 `series1` 的第 1、2 和第 3、4 项，因此答案为 `[1, 1]`，`[2, 3]`。

`[next(series2) for _ in range(2)]` 随后输出两项 `series2`。由于 `series2` 是对 `series1` 做 `filter`，我们要从 `series1` 的下一项 `5` 开始，找两个奇数，也就是 `[5, 8, 13]` 中的 `[5, 13]`。

最后输出 `series1` 的下一项，当前 `series1` 作为斐波那契数列已经生成到了 `13`，因此下一项是 `8 + 13 = 21`。

## 题 2 - Environment Diagram

**评分标准**

1. 每空 0.5 分，箭头和内容一起算一个空；
2. 像 `__a__` `__2__` 这样的变量名和值的 pair，只有变量名对了，后面的值才有分；
3. 分数总是舍入到整数，为了不鼓励胡乱作答，舍入规则为，多写向下舍入，没有多写则向上舍入；
4. 满分必须全对且不能多写，否则只能获得 9 分。

## 题目

```python
def f(g):
    a = 1
    def h(b):
        nonlocal a
        a = 2
        return g(a) * b
    return lambda c: h(c + a)

f(lambda a: a + 3)(4)
```

## 答案

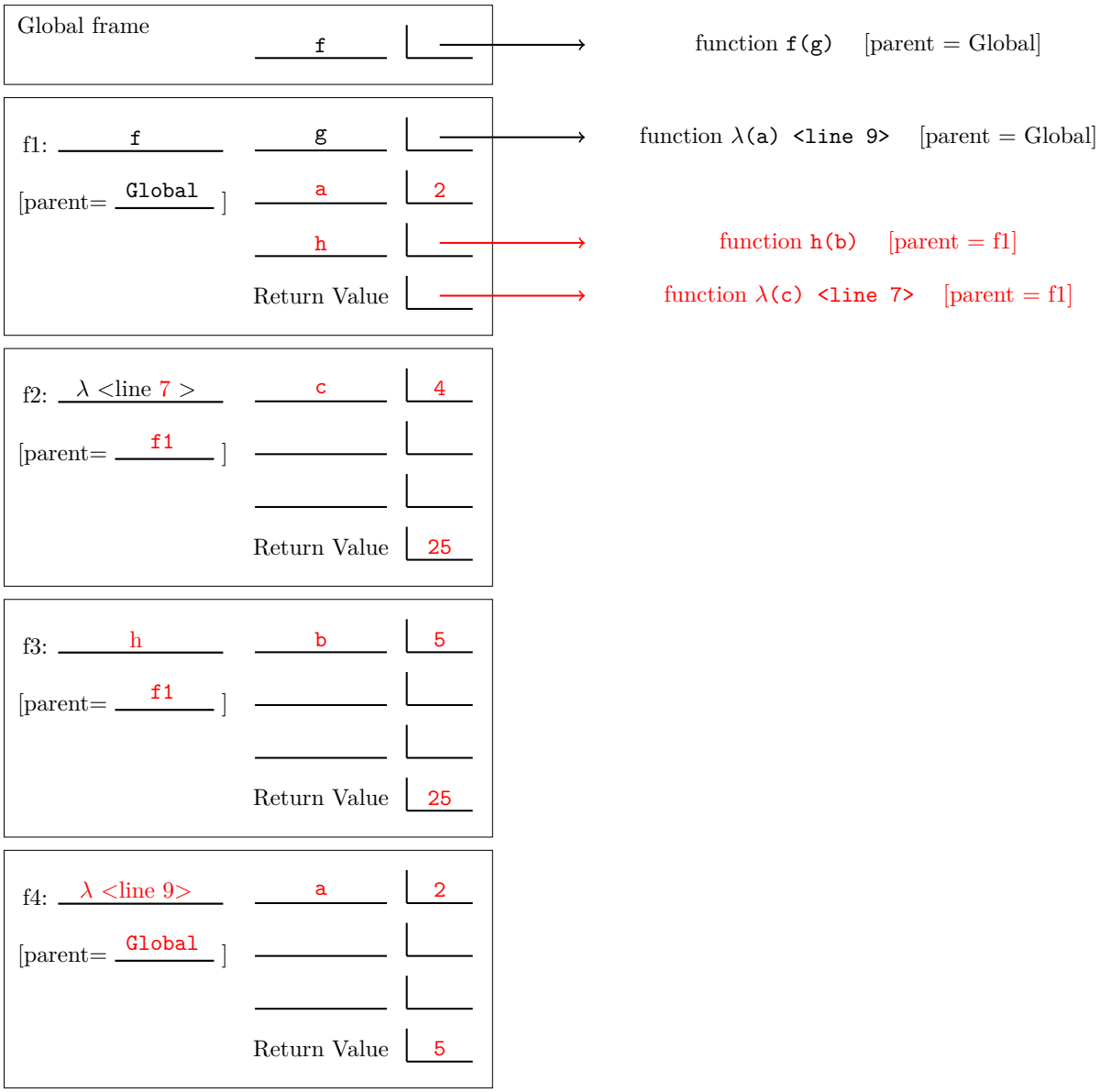



Figure 2: Environment Diagram 答案